

Prefix Completion Using Derivatives

Aditya Gudimetla

August 2026

Abstract Given a context-free grammar Γ and a fragment w that is a prefix of some unknown sentence of Γ , what are the possible completions of w ? Henriksen et al.’s derivative grammars [1] already builds, for every non-terminal A , a derivative A_w whose language is exactly the set of completions of w under A . Checking every non-terminal rather than only the start symbol for shortest completion length rather than nullability yields the non-terminals that can cover w together with the least completion for each, at no cost beyond their construction.

1 Introduction

Derivative grammars are constructed by iteratively differentiating a grammar Γ by an input string w , transforming it into the derivative grammar Γ_w . Nullability of Γ_w ’s start symbol is taken as the test for whether $w \in L(\Gamma)$.

The same differentiation defines an A_w for *every* non-terminal A , not just the start symbol, and $L(A_w)$ is exactly the set of completions of w under A . Thus A covers w iff A_w generates any string.

From the obtained non-terminals, we focus on the minimal covering non-terminals i.e. those whose shortest completion of w is smallest. We find these with a single fixpoint over Γ_w that measures each A_w ’s shortest completion length. Henriksen et al.’s nullability check can be seen as a special case where that length is zero.

2 Background: derivative grammars

Henriksen et al. [1] show that reading a terminal t from a language $L(\Gamma)$ corresponds to rewriting the grammar Γ into a derivative grammar Γ_t generating the derivative language $D_t(L(\Gamma)) = \{x : tx \in L(\Gamma)\}$, via a small set of inference rules over productions. Iterating over a string $w = t_1 \dots t_n$ produces the iterated derivative grammar Γ_w , whose (subscripted) start symbol represents “the rest of the language” after w has been read. For complete input, acceptance is nullability of that symbol, $w \in L(\Gamma) \Leftrightarrow \text{nullable}(S_w)$.

We consider the case where w is a genuine prefix, not malformed input. Two observations, both immediate from this construction, answer the fragment question without further machinery.

Observation 1. For a prefix fragment, the relevant test is not nullability but whether S_w is generating. w is a prefix of some sentence of Γ iff S_w generates some terminal string. More than a decision procedure, $L(S_w) = D_w(L(\Gamma))$ is the set of valid completions of w .

Observation 2. Henriksen et al.’s inference rules are not specific to the start symbol. Every production of every non-terminal is differentiated at every step, so a candidate subscripted non-terminal A_w exists for every $A \in N$. Some of these survive to produce a non-empty language, while others die partway when a terminal of w mismatches every alternative available at

that point, and the corresponding subscripted non-terminal simply never receives a production. Asking “what is w the beginning of” is answered by checking, for every A , whether its already-computed derivative A_w is generating.

3 Definitions

Let $\Gamma = (N, \Sigma, P, S)$ be a CFG and a prefix fragment $w = t_1 \dots t_n$. Build the iterated derivative grammar Γ_w as in Henriksen et al.’s Definition 3.3.

Least possible completion. For a non-terminal A , let

$$\text{LPC}(A, w) = \left\{ \beta \in L(A_w) : |\beta| = \min_{\beta' \in L(A_w)} |\beta'| \right\}.$$

$\text{LPC}(A, w)$ is the set of shortest completions of w under A . We define the shortest completion as a string in $L(A_w)$ of minimum length. Since several completions can share that minimum length, $\text{LPC}(A, w)$ is in general a set, and all of its members have the same length. We call that shared length $m(A)$, the number of terminals needed to complete w as an A .

Covering. A covers w if $A \xRightarrow{*} \alpha w \beta$ for some α, β , that is if w appears in some string A derives. This paper treats the prefix case $\alpha = \varepsilon$, where A covers w exactly when either equivalent condition holds:

1. $L(A_w) \neq \emptyset$
2. $\text{LPC}(A, w) \neq \emptyset$

In either case w is a valid prefix of something A generates. When A does not cover w , this shows up in one of two observationally identical ways:

1. A_w never receives a production, or
2. A_w receives productions but generates nothing.

Covering set and minimum covering set.

Let us define the covering set as

$$\text{Cov}(w) = \{A \in N \mid A \xRightarrow{*} \alpha w \beta\}$$

where $\alpha, \beta \in (N \cup \Sigma)^*$

Taking $\alpha = \varepsilon$ (nothing to the left of w), we get the prefix covers for w , where

$$\text{PrefixCov}(w) = \{A \in N \mid A \xRightarrow{*} w \beta\}$$

is the set of non-terminals for which w is a prefix of some string A derives.

We can now define a $\text{MinPrefixCover}(w)$ as

$$\{A \in \text{PrefixCov}(w) : m(A) = m_{\min}\}$$

where $m_{\min} = \min_{B \in \text{PrefixCov}(w)} m(B)$ is the smallest completion length among all prefix covers. $\text{MinPrefixCover}(w)$ keeps every cover that can generate w and has length $m_{\min}$, since they are all valid completions.

Consequently, if A_w is nullable then $m(A) = 0$, and since a completion length is never negative, such an A is always in $\text{MinPrefixCover}(w)$, as w can be automatically reduced to a complete A . Because every $m(A)$ is the length of an actual derivation $A \xRightarrow{*} w \beta$, $\text{MinPrefixCover}(w)$ never contains an invalid cover.

4 Computing $m(A)$

We compute every $m(A)$ by a shortest-length fixpoint over Γ_w ’s productions, assigning each non-terminal the length of the shortest string it derives. A production costs the sum of its children, a terminal counting one and a non-terminal its own m . $m(A)$ is the smallest such cost over A ’s productions, and we iterate until no length changes.

Example. Take the recursive, ambiguous grammar

$$E \rightarrow E + E \mid E * E \mid (E) \mid C \mid \text{num} \mid \text{id}$$

$$C \rightarrow \text{id} (L)$$

$$L \rightarrow E \mid L, E$$

where C is a call and L an argument list. E is ambiguous (two ways to combine) and recursive (it reaches itself through C and L), yet the fixpoint still converges. Each non-terminal resolves once some production has all its children, giving

$$m(E) = 1 \quad (E \rightarrow \text{num or } E \rightarrow \text{id})$$

$$m(L) = m(E) = 1$$

$$m(C) = 1 + 1 + m(L) + 1 = 4$$

The recursive alternatives $E + E$, $E * E$ and (E) all cost 3, so the minimum never climbs and the iteration halts. Recursion and ambiguity do not affect the shortest length, only the parse forest.

Henriksen et al. consider nullability for the decision problem of strings in the language, which can be considered as the special case where $m(A) = 0$. A non-terminal is nullable exactly when the shortest string it derives is empty.

A non-terminal gets a finite m precisely when it derives some terminal string. The fixpoint terminates because Γ_w has only finitely many subscripted non-terminals and the lengths are strictly decreasing. The subscripts range over the suffixes of w , out of which we only consider those with subscript w as part of the covering set.

5 Worked example

5.1 Grammar 1

Take the toy grammar.

$$S \rightarrow \text{NP VP}$$

$$\text{VP} \rightarrow \text{cl } v \text{ NP}$$

$$\text{NP} \rightarrow \text{det } n$$

5.1.1 Input 1 ($w = \text{"det } n\text{"}$)

$$S_{\text{det}} \rightarrow \text{NP}_{\text{det}} \text{ VP}$$

$$\text{NP}_{\text{det}} \rightarrow n$$

$$S_{\text{det } n} \rightarrow \text{NP}_{\text{det } n} \text{ VP} \implies m(S) = 4$$

$$\text{NP}_{\text{det } n} \rightarrow \varepsilon \implies m(\text{NP}) = 0$$

VP's own production starts with "cl", which does not match "det", so VP_{det} is never generated and $\text{VP} \notin \text{PrefixCov}(\text{det } n)$.

$$\text{PrefixCov}(\text{det } n) = \{\text{NP}, S\}$$

$$\text{MinPrefixCover}(\text{det } n) = \{\text{NP}\}$$

"det n" is exactly a complete NP, while S also covers it, at four extra synthesised terminals.

5.1.2 Input 2 ($w = \text{"cl } v\text{"}$)

A covering set with no nullable member at all.

$$\text{VP}_{\text{cl}} \rightarrow v \text{ NP}$$

$$\text{VP}_{\text{cl } v} \rightarrow \text{NP} \implies m(\text{VP}) = 2$$

(NP has no epsilon production). $S_{\text{cl}} \rightarrow \text{NP}_{\text{cl}} \text{ VP}$, but NP has no production starting with "cl", so NP_{cl} receives zero productions, S_{cl} can never bottom out, and $S \notin \text{PrefixCov}(\text{cl } v)$.

$$\text{PrefixCov}(\text{cl } v) = \{\text{VP}\}$$

$$\text{MinPrefixCover}(\text{cl } v) = \{\text{VP}\}$$

Here $m(\text{VP}) = 2$, and no non-terminal in this covering set is nullable at all.

5.2 Grammar 2

Now take a small expression grammar.

$$\text{Stmt} \rightarrow \text{Expr} ; \mid \text{Lvalue} = \text{Expr} ;$$

$$\text{Expr} \rightarrow \text{id} \mid \text{num} \mid \text{Expr} + \text{Expr}$$

$$\text{Lvalue} \rightarrow \text{id} \mid \text{id} [\text{Expr}]$$

5.2.1 Input 1 ($w = \text{"id"}\text{"}$)

A tie, with two non-terminals already complete.

$$\text{Expr}_{\text{id}} \rightarrow \varepsilon \implies m(\text{Expr}) = 0$$

$$\text{Lvalue}_{\text{id}} \rightarrow \varepsilon \implies m(\text{Lvalue}) = 0$$

$$\text{Stmt}_{\text{id}} \rightarrow ; \implies m(\text{Stmt}) = 1$$

$$\text{PrefixCov}(\text{id}) = \{\text{Expr}, \text{Lvalue}, \text{Stmt}\}$$

$$\text{MinPrefixCover}(\text{id}) = \{\text{Expr}, \text{Lvalue}\}$$

A lone “id” is both a complete expression and a complete lvalue, so both tie at $m = 0$. “Stmt” also covers “id” but needs one more terminal, so it isn’t included in the minimum prefix cover.

6 Conclusions

Given a context-free grammar and a prefix fragment w , the derivative-grammar construction already builds a derivative A_w for every non-terminal A , while only the nullability test for language membership is restricted to the start symbol. Applying instead

1. that test to every A_w , not only the start symbol’s, and
2. a shortest-completion length in place of plain nullability,

yields the non-terminals that can cover w , together with the least possible completion for each, at no cost beyond Henriksen et al.’s existing machinery.

So enumerating the non-terminals that can cover a prefix fragment, ranked by how little each is missing, is a simple process due to the construction of derivative grammars.

Bibliography

- [1] I. Henriksen, G. Bilardi, and K. Pingali, “Derivative Grammars: A Symbolic Approach to Parsing with Derivatives,” *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, 2019, doi: 10.1145/3360553.